Optimal Precision Formats for A100

https://gemini.google.com/app/b5e99fd7704c0d24

User prompt: What is the best precision format for the A100 GPU on Google Colab? I am talking about either the 40 GB or the 80 GB variant. I have access to it on the Colab plan.

Response: For the NVIDIA A100 (Ampere architecture, compute capability 8.0) on Google Colab, **BF16 (Bfloat16)** is the undisputed gold standard for training and fine-tuning, while **TF32** is the best drop-in replacement for standard single precision.

Hardware support, performance, and workload recommendations vary across precision formats:

Precision Comparison on A100

Format	Exponent / Mantissa	Dynamic Range	Hardware Acceleration on A100	Best Use Case
BF16 (`bfloat16`)	8 bits / 7 bits	Same as FP32 (10^{-38} to 10^{38})	Native Tensor Cores (312 TFLOPS)	**LLM training, fine-tuning (LoRA/QLoRA), stable diffusion**
FP16 (`float16`)	5 bits / 10 bits	Narrow (10^{-5} to 6.5×10^{4})	Native Tensor Cores (312 TFLOPS)	Inference, vision models, legacy mixed precision
TF32 (`tensorfloat32`)	8 bits / 10 bits	Same as FP32 (10^{-38} to 10^{38})	Native Tensor Cores (156 TFLOPS)	General PyTorch code without explicit mixed-precision casts
INT8 / INT4	8 / 4 bits integer	Discrete	Tensor Cores (INT8: 624 TOPS)	Quantized inference / QLoRA base weights (`bitsandbytes`, AWQ)
FP8 (`e4m3` / `e5m2`)	8-bit float	Varies	**Not natively supported** (emulated / slow)	Avoid on A100 (requires Ada/Hopper/Blackwell)

Why BF16 Wins for Training & Fine-Tuning

1. **No Gradient Overflow/Underflow:** BF16 preserves the full 8-bit dynamic range of FP32. Unlike FP16, you do not need dynamic loss scaling (`torch.cuda.amp.GradScaler`), eliminating NaN spikes during loss computation and backpropagation.
2. **Identical Throughput to FP16:** Ampere's 3rd-generation Tensor Cores process BF16 and FP16 at the exact same compute rate (up to 312 TFLOPS, or 624 TFLOPS with structural sparsity).
3. **50% Memory Footprint vs FP32:** BF16 cuts parameter and activation memory consumption in half, allowing for substantially larger batch sizes or context lengths on both the 40 GB and 80 GB instances.

The Role of TF32 (TensorFloat-32)

TF32 is internal to Ampere Tensor Cores. It operates on 19-bit representations (using FP32 range and FP16 precision) under the hood while accepting standard 32-bit floats as inputs and producing 32-bit outputs.

In PyTorch, enable TF32 globally across your notebooks for non-mixed precision operations to gain a ~6x to 8x speedup over standard FP32 math with virtually zero loss in numerical stability:

```
import torch

# Enable TF32 for matrix multiplications and convolutions
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True
```

What to Avoid on the A100

- **Native FP8:** The A100 does **not** have native FP8 Tensor Cores. Native FP8 hardware arrived with the Hopper (H100) and Ada Lovelace (RTX 4090/L4) architectures. Running FP8 kernels on an A100 causes software emulation or upcasting, which degrades performance rather than speeding it up.

- **Pure FP32:** There is almost no scenario on an A100 where FP32 should be used end-to-end, as it leaves the Tensor Cores vastly underutilized.

Practical Colab Configurations

- **Hugging Face / Accelerate / PyTorch Training:**

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    bf16=True,      # Use native BF16 mixed precision
    tf32=True,      # Let any remaining FP32 linear layers use TF32
    per_device_train_batch_size=4,
    ...
)
```

- **QLoRA Fine-Tuning on 40GB/80GB:**

```
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,  # Compute activations in BF16
    bnb_4bit_use_double_quant=True
)
```

Sources

1. NVIDIA A100 | Tensor Core GPU

User prompt: Yes, but then again, what is the precision format I should use for maximum token generation? These are the standard precision formats you're talking about. What will maximize token generation?

Response: To maximize **token generation speed (throughput and latency)** on an NVIDIA A100 (40 GB or 80 GB), the optimal precision depends directly on whether you are bound by **single-stream latency** (batch size = 1) or **multi-user throughput** (high concurrency / large batch sizes).

The Short Answer

- **For maximum tokens/sec per stream (Batch Size = 1 to 4):**

  INT4 (AWQ or GPTQ with Marlin/ExLlamaV2 kernels) is the fastest format. It cuts weight transfer time over HBM memory by up to $70–75\%$, delivering $1.8\times$ **to** $2.5\times$ **higher generation speed** than FP16/BF16.

- **For maximum aggregate throughput across concurrent requests (Large Batches):**

  INT8 weight-activation (W8A8) or **AWQ INT4 with FP16 KV-cache** gives the highest ceiling. A100 features dedicated **INT8 Tensor Cores (624 TOPS)**, which operate at $2\times$ the compute throughput of its FP16/BF16 Tensor Cores (312 TFLOPS).

- **Avoid FP8 entirely:** The A100 lacks native FP8 Tensor Cores. Emulated FP8 on an A100 will be drastically slower than INT8 or FP16.

Why Token Generation Bottlenecks Differ

LLM generation has two distinct computational phases:

1. **Autoregressive Decoding (Token Generation):** Each new token requires streaming **all model weights from HBM to the Tensor Cores**. This phase is almost purely **memory-bandwidth bound**.
2. **Prefill (Prompt Processing):** Highly parallelized across tokens; this phase is **compute-bound**.

Because token generation is bottlenecked by the A100's memory bus ($1,555$ GB/s on 40 GB PCIe; $2,039$ GB/s on 80 GB SXM), the fewer bytes you fetch per weight, the faster tokens are produced:

$$\text{Theoretical Speedup} \approx \frac{\text{Bytes per parameter in FP16 (2 bytes)}}{\text{Bytes per parameter in Target Format}}$$

Format Breakdown for Inference Generation

Quantization Format	Bits per Weight / Activation	Primary Engine / Kernel	Relative Speed (Batch Size = 1)	KV Cache Overhead	Notes
AWQ / GPTQ (W4A16)	4-bit weights / 16-bit activations	**vLLM (Marlin)** or **TRT-LLM**	$\mathbf{2.0\times - 2.5\times}$	FP16 / BF16	**Fastest single-stream speed**. Weights are decompressed on-the-fly into 16-bit registers.
SmoothQuant / INT8 (W8A8)	8-bit weights / 8-bit activations	**TensorRT-LLM** / **vLLM**	$\mathbf{1.5\times - 1.8\times}$	INT8 / FP16	Highest compute ceiling for high-batch serving (624 TOPS).
BitsAndBytes (NF4)	4-bit weights / 16-bit activations	Hugging Face Transformers	$\mathbf{0.6\times - 0.8\times}$	FP16 / BF16	**Slow generation.** Great for training/QLoRA memory savings, but lacks optimized dequantization inference kernels.
FP16 / BF16	16-bit / 16-bit	FlashAttention-2 / vLLM	$1.0\times$ **(Baseline)**	FP16 / BF16	Zero accuracy loss, but bound to reading 2 bytes per weight per token.

Optimal Implementation on Colab

To achieve high token throughput on an A100, standard Hugging Face pipelines (`generate()`) should be avoided, as their Python loops add high overhead. Instead, use an engine with fused attention and dequantization kernels:

1. Maximum Throughput via vLLM + AWQ (Marlin Kernel)

AWQ with the Marlin kernel executes 4-bit dequantization directly inside Ampere Tensor Cores:

```
!pip install vllm

from vllm import LLM, SamplingParams

# Load pre-quantized 4-bit AWQ model
llm = LLM(
    model="Qwen/Qwen2.5-7B-Instruct-AWQ",  # or any AWQ/GPTQ model
    quantization="marlin",                # Marlin utilizes A100 memory bandwidth optimally
    dtype="float16",
    max_model_len=4096,
    gpu_memory_utilization=0.90
)

sampling_params = SamplingParams(temperature=0.7, max_tokens=512)
outputs = llm.generate(["Explain quantum entanglement simply:"], sampling_params)
print(outputs[0].outputs[0].text)
```

2. Maximize KV Cache Capacity for Long Generations

If running long context or batch generation on the A100, the KV cache can quickly eat into HBM. Quantizing the KV cache to **FP8** (`e4m3` or `e5m2`) or **INT8** frees up VRAM for higher batch concurrency:

```
# In vLLM, compress the KV cache to 8-bit without degrading weight speed
llm = LLM(
    model="TheBloke/Llama-2-13B-chat-AWQ",
    quantization="awq",
    kv_cache_dtype="fp8"  # Supported in vLLM software cache even on Ampere
)
```

Recommendation

- **For maximum tokens/sec right now:** Use **AWQ (4-bit)** powered by **vLLM with Marlin kernels**.
- **If accuracy degradation on 4-bit is a concern (e.g., complex math/code):** Use **INT8 (W8A16 or W8A8)** via TensorRT-LLM or vLLM.
- **Avoid standard `load_in_4bit=True` (bitsandbytes) for generation speed:** It cuts VRAM for training, but its inference dequantization kernel is memory-inefficient and will run noticeably slower than FP16.

Sources

1. LLM Quantization: BF16 vs FP8 vs INT4 - AIMultiple
